

Politecnico di Milano

Anno accademico 2013-2014

Ingegneria del Software – Appello del 23 luglio 2014

Cognome:		
Nome:		Matricola:
Sezione (segnarne una): ☐ Baresi	☐ Ghezzi	☐ San Pietro

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare solo i fogli distribuiti utilizzando il retro delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. non verranno in nessun caso presi in considerazione. È possibile scrivere in matita.
3. È possibile consultare liberamente libri, manuali o appunti. È proibito l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, ecc.).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. Tempo a disposizione: 2h.

Esercizio 1:

Esercizio 2:

Esercizio 3:

Esercizio 4:

Esercizio 5:

Esercizio 1

Durante la realizzazione di un gioco multi-giocatore, occorre definire una struttura dati, adatta a rappresentare un labirinto. Il labirinto è costituito da stanze, collegate da porte magiche. Vi è una stanza di ingresso al labirinto e un certo numero di stanze di uscita dallo stesso. Ogni stanza ha un nome (una stringa) ed è dotata di almeno una porta per passare istantaneamente in una stanza diversa. Anche ogni porta ha un nome; se da una stanza x esiste una porta verso una stanza y, allora questa porta è unica.

Il labirinto in Java è rappresentato da una classe immutabile, i cui esemplari sono costruiti a partire da un file opportuno, caratteristica che per semplicità qui ignoriamo. Di seguito è riportata la specifica delle classi rilevanti.

```
public /*@ pure @*/ class Stanza {

   //s e' l'insieme delle porte in uscita a this
   public Stanza(Set<Porta> s, String nome);

  //@ensures (* \result e' l'insieme di porte in uscita dalla stanza this*);
  public Set<Porta> porteOut();

  //@ensures (* \result e' il nome della stanza this*);
  public String nome()
}

public /*@ pure @*/ class Porta {

   //x e' la stanza verso cui si viene trasportati
   public Porta(Stanza x, String nome)

  //@ensures (* \result e' la stanza a cui si viene trasportati passando attraverso this.*);
  public Stanza versoStanza();

  //@ensures (* \result e' il nome della porta this*);
  public String nome()

}

public /*@ pure @*/ class Labirinto {

  public Labirinto(...strutture adeguate a caricare un labirinto non vuoto...);

  //@ensures (* \result e' true se, e solo se, esiste una porta che
  //@           collega direttamente la stanza x alla stanza y*);
  public boolean esistePorta(Stanza x, Stanza y)

  //@ensures (* \result e' la stanza di ingresso al labirinto*)
  public Stanza ingresso()

  //@ensures (* \result e' l'insieme di tutte le stanze del labirinto*)
  public Set<Stanza> stanze()

  //@ensures (* \result e' il sottoinsieme di stanze() che sono di uscita dal labirinto*)
  public Set<Stanza> uscite()

  //@ensures (*\result restituisce un iteratore che itera su tutte le porte di this,
  //@           una alla volta, in ordine qualunque*);
  Iterator<Porta> porte()
}
```

Si rammenta che un Set è una collection di Java, in cui non vi possono essere elementi duplicati. Si può accedere agli elementi di un set esclusivamente tramite i consueti metodi `contains`, `containsAll`, `isEmpty` e `size`.

Domanda 1 Specificare in JML il metodo `esistePorta`.

SOLUZIONE: Ipotizziamo, qui e nel seguito, che i metodi delle varie classi non ritornino mai il valore null, ne' null sia un valore ammissibile per una porta o una stanza del labirinto.

```
 //equires stanze().contains(x) && stanze().contains(y);
//@ensures \result ==
//@        (\exists Porta p; x.porteOut().contains(p); p.versoStanza().equals(y));
```

Domanda 2 Si vuole definire un `LabirintoStregato`, simile al `Labirinto`, ma in cui alcune delle `Porte` sono rimpiazzate da una `PortaStregata`. Una `PortaStregata` è identica a una `Porta`, salvo che la stanza a cui conduce è scelta casualmente fra tutte le stanze. Le classi `PortaStregata` e `LabirintoStregato` sono definite nel modo seguente:

```
public class PortaStregata extends Porta {
  public PortaStregata(Set<Stanza> stanze, String nome);
  @override
  //@ensures (*\result e' una stanza scelta a caso *);
  public Stanza versoStanza();
}

public class LabirintoStregato extends Labirinto {
 // ridefinizione del solo costruttore, che permette di
 // costruire un labirinto in cui una o piu' porte sono
 // stregate
}
```

Queste ridefinizioni verificano il principio di sostituzione? Giustificare la risposta.

SOLUZIONE:

La classe PortaStregata verifica la regola dei metodi, in quanto la postcondizione garantisce di restituire una stanza, che e' tutto quello che viene promesso dalla postcondizione originale. Tuttavia, la regola delle proprietà è violata, in quanto la classa PortaStregata diventa mutabile, ossia non è piu' pura: la stanza restituita e' un invariante della classe originale. La classe LabirintoStregato non viola naturalmente la regola dei metodi, ma anch'essa viola la regola delle proprieta'. Ad esempio, l'invariante che non ci sono in una stanza due porte che conducono a una stessa stanza, puo' essere violato, in quanto in modo causale in certi momenti due porte potrebbero riferirsi alla stessa stanza.

Domanda 3 Si consideri una generalizzazione del concetto di `Porta`, ipotizzando che ogni porta possa avere anche un effetto sulle caratteristiche dell'avatar che la attraversa. Per esempio, una `Porta Magica` consente all'avatar di assumere alcuni poteri magici per un certo tempo; una `Porta Premio` incrementa il punteggio del giocatore; una `Porta Resurrezione` fornisce una vita supplementare al giocatore; una `Porta Maledizione` procura una maledizione che rispedisce il giocatore a un livello precedente, ecc. Il tipo delle porte potrebbe cambiare a *run-time* e nuovi tipi di porta potrebbero essere definiti in base al particolare tipo di gioco che usa il labirinto. Quale potrebbe essere l'organizzazione di una struttura a classi che consente di realizzare un progetto flessibile ed estendibile? Non serve scrivere il codice Java, ma un diagramma delle classi UML potrebbe essere utile per rappresentare le diverse classi e le loro relazioni.

SOLUZIONE:

Si può utilizzare una Strategy. Supponendo che vi sia una classe Avatar, che descrive l'avatar di un giocatore, si potrebbe definire l'interfaccia:

```
interface ComportamentoPorta {
  //@ensures (* agisce sull'avatar p *);
  public void effetto(Avatar p)
}
```

la classe Porta potrebbe essere così definita:

```
public class Porta {
  //questi metodi non cambiano:
  public Porta(Stanza x, String nome, ComportamentoPorta p)
  public Stanza versoStanza();
  public String nome();
  //un pezzo del rep:
  private ComportamentoPorta comportamento;
  //nuovo metodo per associare un comportamento
  public associaEffetto(ComportamentoPorta p) {
     this.comportamento == p;
 }
}
```

Ad esempio, il comportamento della Porta Premio potrebbe essere ottenuto così:

```
public class ComportamentoPortaPremio implements ComportamentoPorta {
  //@ensures (* incremento il punteggio di p *)
  public void effetto (Avatar p);
}
```

Il pattern Strategy consente di modificare a run-time il comportamento della Porta, senza modificare la classe Labirinto. Un Abstract per la gerarchia delle porte non consente di cambiare facilmente il comportamento delle porte: occorre ogni volta una nuova istanza di Porta, copiarvi le altre caratteristiche della Porta esistente e infine memorizzare nel Labirinto la nuova porta al posto di quella vecchia (aggiungendo naturalmente i metodi opportuni per farlo): la struttura e' piu' goffa e poco flessibile.

Un Decorator puo' svolgere lo stesso ruolo di una Strategy, ma e' eccessivamente generale per risolvere questo problema particolare: un decorator e' utile quando ci sono piu' attributi/metodi che possono variare, anche a livello dell'interfaccia della classe, mentre lo Strategy e' da preferire quando si deve scegliere fra diverse implementazioni di un'operazione (a parita' di interfaccia).

Esercizio 2

Sia data per la classe `Labirinto` la seguente implementazione. Il numero totale di stanze del labirinto è memorizzato nel campo intero `numStanze`. Le stanze sono rappresentate da una map `mappaStanze`, che associa ad ogni stanza un intero progressivo (in un ordine arbitrario), partendo da 0. Lo 0 corrisponde sempre alla stanza di ingresso.

Il labirinto è rappresentato con una matrice quadrata `collegamenti`, di dimensione `numStanze` $\times$ `numStanze`. Nella posizione (i, j) della matrice è memorizzata, se esiste, la porta che collega la stanza di numero i con la stanza di numero j; se la porta non esiste, allora è memorizzato il valore `null`.

```
private int numStanze;
private Map<Stanza,Integer> mappaStanze;
private Porta[][] collegamenti;
```

Si rammenta che l'interfaccia `Map<K,V>` prevede, fra l'altro, i seguenti metodi puri. Il metodo `V get(Object x)`, dato un `x` di tipo `K`, restituisce l'elemento di tipo `V` associato a `x`, se questo esiste (se non esiste, get restituisce `null`). Quindi se p.es. `x` è la stanza di ingresso, `get(x)` restituisce l'intero 0, ecc. Il metodo `size()` restituisce la dimensione della mappa (ossia il numero complessivo di stanze). Il metodo `keySet()` restituisce un Set che contiene tutte le chiavi (ossia tutte le stanze memorizzate nella mappa). Infine, il metodo `values()` restituisce una `Collection<V>` che contiene tutti i valori della mappa.

Domanda 1 Scrivere l'invariante di rappresentazione. Facoltativamente, si scriva anche la funzione di astrazione (utilizzando la tecnica preferita) della classe `Labirinto`.

SOLUZIONE:

```
//RI:
private invariant collegamenti!= null && mappaStanze!= null &&
//numStanze e' proprio la dim. della matrice collegamenti:
  numStanze == collegamenti.length &&
//la matrice collegamenti e' quadrata:
  (\forall int i; 0<=i && i<numStanze; collegamenti[i].length == numStanze) &&
//mappaStanze restituisce valori compresi fra 0 e numStanze:
 (\forall Integer x;; mappaStanze.values().contains(x) <==> 0<= x && x< numStanze) &&
//mappaStanze non contiene null:
!mappaStanze.keySet().contains(null);
```

Si puo' descrivere anche la funzione di astrazione con un invariante privato, che correla i campi privati con i metodi osservatori:

```
private invariant
//il numero di stanze corrisponde:
  numStanze == stanze().size() &&
//l'insieme di stanze della mappa e' proprio l'insieme di stanze del labirinto:
  mappaStanze.keySet().equals(stanze()) &&
//l'ingresso ha valore 0:
  mappaStanze.get(ingresso()) == 0 &&
//le porte nei collegamenti sono quelle corrette:
  (\forall Stanza x; stanze().contains(x);
     (\forall Stanza y; stanze().contains(x) && x!= y;
         esistePorta(x,y) <==> collegamenti[mappaStanze().get(x)][mappaStanze().get(y)]!=null))
```

Domanda 2 Implementare l'iteratore `porte()`.

Esercizio 3

Si consideri il seguente programma

```
public static int gcd(int a, int b) { //valid for positive integers.
   while (a !=b) {
      if (a > b) a = a - b;
      else b = a - b;
   }
   return a;
}
```

1. Scrivere la condizione logica che deve essere soddisfatta dai valori in input (chiamati `a` e `b`) affinchè venga testato il cammino del programma che esegue il loop due volte, la prima eseguendo il ramo then e la seconda eseguendo il ramo else
2. Qualora la condizione sia soddisfacibile, trovare valori interi che causano l'esecuzione di questo cammino.
3. Si supponga di eliminare dal programma il ramo `else` dell'`if`. Definire per questo programma casi di test che coprano tutti i branch. Mostrare l'effetto dell'esecuzione del programma e la capacità di trovare errori mediante l'esecuzione dei programma per i casi di test.

SOLUZIONE (1) e (2): La condizione logica e' l'AND di `b<a<2b` (condizione per eseguire prima il then e poi l'else) e di `b==0` (condizione per uscire dal ciclo dopo due iterazione). La condizione e' ovviamente insoddisfacibile. Di fatto, e' possibile selezionare valori per percorrere il cammino richiesto (p.es. `a=3, b=2`), ma poi si entra per forza in loop infinito. (3): Un caso di test deve soddisfare `a>b>0` per potere entrare almeno una volta nel while e nel ramo then, con la condizione che a sia multiplo di b per potere uscire dal ciclo. P.es. `a=2, b=1`: il programma esegue una volta il while, poi il ramo then: a quel punto `a=b=1` e il programma esce; un altro esempio é `a=6, b=2`, che resta tre volte nel ciclo, ecc.

Un caso di test che copre l'altro ramo dell'if deve verificare `a<b`, ad esempio `a=2, b=3`). L'esecuzione con `a<b` entra in loop infinito (in quanto ne' a ne' b sono modificati). Quindi il testing co il criterio di copertura delle diramazioni in questo caso e' in grado di rilevare un errore significativo, che poteva non essere rilevato ad esempio con il criterio di copertura delle istruzioni (il test con `a=2, b=1` ad esempio copre tutte le istruzioni, ma non rileva l'errore).

Esercizio 4

Si progetti una classe Java `Contatore` che permetta l'accesso concorrente ad un contatore che:

- Inizialmente vale 0 e può arrivare a 10;
- Quando il contatore arriva a 10, il metodo `reset()` consente di tornare a 0. L'operazione di reset deve essere possibile solo se il contatore vale 10. I metodi `incrementa()` e `decrementa()` consentono l'incremento e il decremento di una unità per volta. Le operazioni di incremento e decremento devono sempre mantenere il valore del contatore tra 0 e 10.

Suggerimento: si sfrutti opportunamente il modificatore `synchronized` per evitare inconsistenze nell'accesso all'oggetto condiviso.

SOLUZIONE

Ipotizziamo che quando il contatore sia arrivato a 10, l'eventuale incremento si sospenda fino a quando un decremento o un reset portino il contatore a un valore inferiore. Similmente per il reset e il decremento. La soluzione in questo caso e', ignorando le eccezioni `InterruptedException`:

```
public class Contatore {
  private int c; //il contatore
  public synchronized void incrementa {
    while (c==10) wait();
    c++;
    notifyAll();
  }
  public synchronized void decrementa {
    while (c==0) wait();
    c--;
    notifyAll();
  }
  public synchronized void reset {
    while (c<10) wait();
    c=0;
    notifyAll();
  }
```